



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 7
по курсу «Алгоритмы компьютерной графики»
«Оптимизация приложений OpenGL »

Студент группы ИУ9-42Б Ивенкова О. В.

Преподаватель Цалкович П. А.

Москва 2025

1 Задача

- Оптимизация приложения OpenGL, созданного в рамках предыдущей лабораторной работы, на основе выбора наиболее эффективных методик. (см. Баяковский Ю.М., Игнатенко А.В. Начальный курс OpenGL.- М.: «Планета Знаний», 2007.- 221с. – Глава 9).
- Обязательно использовать дисплейные списки и массивы вершин и еще 2 любые различные оптимизации(в сумме минимум 4 оптимизации).
- Оценка применимости выбранного метода оптимизации приложения OpenGL должна осуществляться на основании измерения производительности.
- Результаты замеров оформить в табличном виде.

2 Основная теория

Организация приложения

- высокоуровневая оптимизация: необходим поиск компромисса между качеством и производительностью, например:
 - отображение геометрии сцены с низким качеством (с уменьшенным количеством примитивов) во время анимации, и с наилучшим качеством (полностью) в моменты остановок;
 - представление объектов, располагающихся далеко от наблюдателя, моделями пониженной сложности
 - объекты, которые находятся полностью вне поля видимости, могут быть эффективно отсечены без передачи на конвейер OpenGL с помощью проверки попадания ограничивающих их простых объемов (сфер или кубов) в объем видимости;
 - во время анимации можно отключить псевдотонирование (dithering), плавную заливку, текстуры;
- низкоуровневая оптимизация: использование структур данных, которые могут быть быстро и эффективно переданы на конвейер OpenGL.

Оптимизация вызовов OpenGL: передача данных

- использование связанных примитивов (**GL_LINES**, **GL_LINE_LOOP**, **GL_TRIANGLE_STRIP**, **GL_TRIANGLE_FAN** и **GL_QUAD_STRIP**);
- использование массивов вершин (**glVertexPointer**, **glNormalPointer**, **glColorPointer**, **glInterleavedArrays**, **glDrawArrays**, **glArrayElement**);
- задание необходимых массивов одной командой (**glVertexPointer / glNormalPointer / glColorPointer** → **glInterleavedArrays**);
- использование индексированных примитивов (**glDrawElements**);
- последовательное хранение в памяти данных о вершинах (влияет на скорость обмена между основной памятью и графической подсистемой);
- использование векторных версий **glVertex**, **glColor**, **glNormal** и **glTexCoord**;
- уменьшение сложности примитивов (при сохранении наилучшего соотношения качества и производительности, в частности, при использовании текстурирования);
- использование дисплейных списков (могут храниться в памяти графической подсистемы);
- исключения указания ненужных атрибутов вершин (например, **glNormal** при выключенном освещении);
- минимизация количества лишнего кода между **glBegin/glEnd**.

Оптимизация вызовов OpenGL: управление обработкой вершин в графическом конвейере

- освещение:
 - исключить использование локальных источников света, т.е. координаты источника должны быть в форме (x,y,z,0);
 - исключить использование двухстороннего освещения (two-sided lighting);
 - исключить использование локальной модели освещения;
 - исключить частую смену параметра материала **GL_SHININESS**;
 - рассмотреть возможность заранее просчитать освещение (можно получить эффект освещения, задавая цвета вершин вместо нормалей);
- отключение нормализации векторов нормалей (glEnable / glDisable(GL_NORMALIZE));
- использование связанных примитивов (**GL_LINES**, **GL_LINE_LOOP**, **GL_TRIANGLE_STRIP**, **GL_TRIANGLE_FAN**, и **GL_QUAD_STRIP**).

Оптимизация вызовов OpenGL: управление растеризацией

- отключение теста на глубину (фоновые объекты, например, могут быть нарисованы без теста на глубину, если они визуализируются первыми);
- отсечение (отбрасывание) обратных граней полигонов до растеризации (glEnable(GL_CULL_FACE));
- минимизация лишних операций с пикселями (маскирование, альфа-смешивание и другие попиксельные операции могут занимать существенное время на этапе растеризации);
- уменьшение размера окна или разрешения экрана (в случае, если меньшие размеры окна или меньшее разрешение экрана приемлемы).

Оптимизация вызовов OpenGL: управление текстурированием

- использование эффективных форматов хранения изображений (**GL_UNSIGNED_BYTE**);
- объединение текстур в текстурные объекты или дисплейные списки (при использовании нескольких текстур позволяет графической подсистеме эффективно управлять размещением текстур в видеопамяти)
- исключение использования текстур большого размера;
- комбинирование небольших текстур в одну большего размера и изменение текстурных координат для работы с нужной подтекстурой (позволяет уменьшить число переключений текстур);
- при использовании анимированных текстур для обновления образа текстуры - вызов команд **glTexSubImage2D** или **glTexCopyTexSubImage2D** вместо **glTexImage2D**.

Дисплейные списки (display lists)

- если несколько раз производится обращение к одной и той же группе команд, то их можно объединить в *дисплейный список*, и вызывать его при необходимости;
- дисплейные списки в оптимальном, скомпилированном виде хранятся в памяти сервера, что позволяет рисовать примитивы в такой форме максимально быстро;
- в то же время большие объемы данных занимают много памяти, что в свою очередь влечет падение производительности (большие объемы (больше нескольких десятков тысяч примитивов) лучше рисовать с помощью массивов вершин);
- работа с дисплейными списками:
 1. создание дисплейного списка: **glNewList()** / **glEndList()**.
 - идентификация списка: целое положительное число
 - режим обработки списка:
 - **GL_COMPILE:** команды записываются в список без выполнения
 - **GL_COMPILE_AND_EXECUTE:** команды сначала выполняются, а затем записываются в список
 2. вызов списка(ов): **glCallList()** / **glCallLists()**
 3. удаление дисплейного списка: **glDeleteList()**

Массивы вершин

- задание массивов:
 - массив координат вершин:
glVertexPointer (GLint size, GLenum type, GLsizei stride, void* ptr)
 - массив координат нормалей:
glNormalPointer (GLenum type, GLsizei stride, void* ptr)
 - массив цветов:
glColorPointer (GLint size, GLenum type, GLsizei stride, void* ptr)
 - массив текстурных координат:
glTexCoordPointer (GLint size, GLenum type, GLsizei stride, void* ptr)
 - одновременно несколько массивов:
glInterleavedArrays (GLenum format, GLsizei stride, const GLvoid * pointer);
- определение используемых массивов:
glEnableClientState (GLenum array) / glDisableClientState (GLenum array)
- использование сформированных массивов для отрисовки:
 - отдельного элемента:
void glVertexArrayElement (GLint i)
 - набора примитивов по данным из массивов:
void glDrawArrays (GLenum mode, GLint first, GLsizei count);
 - набора примитивов по данным из массивов (по индексам):
void glDrawElements (GLenum mode, GLsizei count, GLenum type, const GLvoid * indices);

3 Практическая реализация

Исходный код программы представлен в листинге 1.

Листинг 1: Реализация файла main.py

```
1 import glfw
2 from OpenGL.GL import *
3 from OpenGL.GLUT import *
4 from OpenGL.GLU import *
5 import numpy as np
6 from PIL import Image
7 '''
8 MEASURE_DURATION = 5.0
9 measure_start_time = None
10 frame_count = 0
11 total_render_time = 0.0
12 '''
13 angle_x, angle_y = 0, 0
14 scale = 0.7
15 wireframe = False
16 light_diffuse = [0.2, 0.2, 0.2, 1.0]
17 light_position_toggle = False
18 light_diffuse_toggle = False
```

```

19 light_specular_toggle = False
20
21 gravity = -9.8
22 y_pos = 1.0
23 y_velocity = 2.5
24 last_time = glfw.get_time()
25 use_texture = True
26
27 light_needs_update = True
28
29 vertex_array = None
30 texcoord_array = None
31 index_array = None
32 vao_created = False
33
34 def generate_ellipsoid_arrays():
35     global vertex_array, texcoord_array, index_array
36     vertices = []
37     texcoords = []
38     indices = []
39     for i in range(num_lat + 1):
40         theta = -np.pi / 2 + np.pi * i / num_lat
41         for j in range(num_lon + 1):
42             phi = 2 * np.pi * j / num_lon
43             x, y, z = parametric_ellipsoid(theta, phi)
44             vertices.append((x, y, z))
45             texcoords.append((j / num_lon, i / num_lat))
46
47     for i in range(num_lat):
48         for j in range(num_lon):
49             idx = i * (num_lon + 1) + j
50             idx_next = (i + 1) * (num_lon + 1) + j
51             indices += [
52                 idx, idx_next, idx_next + 1,
53                 idx, idx_next + 1, idx + 1
54             ]
55
56     vertex_array = np.array(vertices, dtype=np.float32)
57     texcoord_array = np.array(texcoords, dtype=np.float32)
58     index_array = np.array(indices, dtype=np.uint32)
59
60
61 display_list_id = None
62
63 def create_ellipsoid_display_list():
64     global display_list_id

```

```

65     display_list_id = glGenLists(1)
66     glNewList(display_list_id, GL_COMPILE)
67     draw_parametric_ellipsoid()
68     glEndList()
69
70 a, b, c = 1.0, 0.6, 0.8
71 num_lat = 10
72 num_lon = 20
73
74 def parametric_ellipsoid(theta, phi):
75     x = a * np.cos(theta) * np.cos(phi)
76     y = b * np.sin(theta)
77     z = c * np.cos(theta) * np.sin(phi)
78     return x, y, z
79
80 def draw_parametric_ellipsoid():
81     glEnableClientState(GL_VERTEX_ARRAY)
82     glVertexPointer(3, GL_FLOAT, 0, vertex_array)
83     if use_texture:
84         glEnableClientState(GL_TEXTURE_COORD_ARRAY)
85         glTexCoordPointer(2, GL_FLOAT, 0, texcoord_array)
86
87     glDrawElements(GL_TRIANGLES, len(index_array), GL_UNSIGNED_INT,
88 index_array)
89
90     glDisableClientState(GL_VERTEX_ARRAY)
91     if use_texture:
92         glDisableClientState(GL_TEXTURE_COORD_ARRAY)
93
94 def draw_moving_ellipsoid():
95     glMatrixMode(GL_MODELVIEW)
96     glLoadIdentity()
97     glRotatef(angle_x, 1, 0, 0)
98     glRotatef(angle_y, 0, 1, 0)
99
100     glPushMatrix()
101     glTranslatef(0, y_pos, 0)
102     glScalef(scale, scale, scale)
103     glColor3f(75/255, 0, 130/255)
104     glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, [0.2, 0.2, 0.4, 1.0])
105     glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, [0.3, 0.3, 0.8, 1.0])
106     glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, [1.0, 1.0, 1.0, 1.0])
107     glMaterialf(GL_FRONT_AND_BACK, GL_SHININESS, 50.0)
108     #draw_parametric_ellipsoid()
109     if display_list_id is not None:
110         glCallList(display_list_id)

```



```

110     glPopMatrix()
111
112 def setup_lighting():
113     glEnable(GL_LIGHTING)
114     glEnable(GL_LIGHT0)
115     glShadeModel(GL_SMOOTH)
116
117     glLightModelfv(GL_LIGHT_MODEL_AMBIENT, [0.5, 0.5, 0.5, 1.0])
118
119     glLightfv(GL_LIGHT0, GL_POSITION, [2.0, 2.0, 2.0, 1.0])
120     glLightfv(GL_LIGHT0, GL_DIFFUSE, [1.0, 1.0, 1.0, 1.0])
121     glLightfv(GL_LIGHT0, GL_SPECULAR, [1.0, 1.0, 1.0, 1.0])
122
123     glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, [0.2, 0.2, 0.4, 1.0])
124     glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, [0.3, 0.3, 0.8, 1.0])
125     glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, [1.0, 1.0, 1.0, 1.0])
126     glMaterialf(GL_FRONT_AND_BACK, GL_SHININESS, 50.0)
127
128 def update_light():
129     global light_position_toggle, light_diffuse_toggle,
130     light_specular_toggle, light_needs_update
131
132     if not light_needs_update:
133         if light_position_toggle:
134             glLightfv(GL_LIGHT0, GL_POSITION, [-2.0, -2.0, 2.0, 1.0])
135         else:
136             glLightfv(GL_LIGHT0, GL_POSITION, [2.0, 2.0, 2.0, 1.0])
137
138         if light_diffuse_toggle:
139             glLightfv(GL_LIGHT0, GL_DIFFUSE, [0.2, 1.0, 0.2, 1.0])
140         else:
141             glLightfv(GL_LIGHT0, GL_DIFFUSE, [1.0, 1.0, 1.0, 1.0])
142
143         if light_specular_toggle:
144             glLightfv(GL_LIGHT0, GL_SPECULAR, [0.0, 0.0, 0.0, 1.0])
145         else:
146             glLightfv(GL_LIGHT0, GL_SPECULAR, [1.0, 1.0, 1.0, 1.0])
147
148     light_needs_update = False
149
150 def update_motion():
151     global y_pos, y_velocity, last_time
152
153     current_time = glfw.get_time()
154     delta_time = current_time - last_time
155     last_time = current_time

```

```

155
156     y_velocity += gravity * delta_time
157     y_pos += y_velocity * delta_time
158
159     if y_pos <= -1.0:
160         y_pos = -1.0
161         y_velocity = -y_velocity
162
163 def load_texture(path):
164     image = Image.open(path)
165     image = image.transpose(Image.FLIP_TOP_BOTTOM)
166     img_data = image.convert("RGB").tobytes()
167
168     texture_id = glGenTextures(1)
169     glBindTexture(GL_TEXTURE_2D, texture_id)
170
171     glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, image.width, image.height,
172                 0, GL_RGB, GL_UNSIGNED_BYTE, img_data)
173
174     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)
175     return texture_id
176
177 def display():
178     '''
179     global measure_start_time, frame_count, total_render_time
180     if measure_start_time is None:
181         measure_start_time = glfw.get_time()
182     frame_start = glfw.get_time()
183     '''
184
185     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
186     draw_moving_ellipsoid()
187     glfw.swap_buffers(window)
188
189     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
190     draw_moving_ellipsoid()
191     glfw.swap_buffers(window)
192     '''
193     frame_end = glfw.get_time()
194     render_time = frame_end - frame_start
195     total_render_time += render_time
196     frame_count += 1
197     '''
198
199 def key_callback(window, key, scancode, action, mods):

```

```

200     global angle_x, angle_y, wireframe, scale, use_texture,
light_position_toggle, light_diffuse_toggle, light_specular_toggle,
light_needs_update
201     if action == glfw.PRESS or action == glfw.REPEAT:
202         if key == glfw.KEY_UP:
203             angle_x -= 5
204         elif key == glfw.KEY_DOWN:
205             angle_x += 5
206         elif key == glfw.KEY_LEFT:
207             angle_y -= 5
208         elif key == glfw.KEY_RIGHT:
209             angle_y += 5
210         elif key == glfw.KEY_EQUAL:
211             scale += 0.1
212         elif key == glfw.KEY_MINUS:
213             scale -= 0.1
214         elif key == glfw.KEY_W:
215             wireframe = not wireframe
216             glPolygonMode(GL_FRONT_AND_BACK, GL_LINE if wireframe else
GL_FILL)
217         elif key == glfw.KEY_T:
218             use_texture = not use_texture
219             if use_texture:
220                 glEnable(GL_TEXTURE_2D)
221             else:
222                 glDisable(GL_TEXTURE_2D)
223         elif key == glfw.KEY_1:
224             light_needs_update = True
225             light_position_toggle = not light_position_toggle
226             update_light()
227         elif key == glfw.KEY_2:
228             light_needs_update = True
229             light_diffuse_toggle = not light_diffuse_toggle
230             update_light()
231         elif key == glfw.KEY_3:
232             light_needs_update = True
233             light_specular_toggle = not light_specular_toggle
234             update_light()
235
236 def scroll_callback(window, xoffset, yoffset):
237     global scale
238     scale += 0.1 if yoffset > 0 else -0.1
239
240 def init():
241     glEnable(GL_DEPTH_TEST)
242     glEnable(GL_TEXTURE_2D)

```

```

243     texture_id = load_texture(r"C:\Go\projects\Bmstu-projects\Grafic\
lab6\texture.bmp")
244     glTexEnvf(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_MODULATE)
245     setup_lighting()
246     glEnable(GL_CULL_FACE)
247     glCullFace(GL_BACK)
248     glFrontFace(GL_CCW)
249     generate_ellipsoid_arrays()
250     create_ellipsoid_display_list()
251
252     if not glfw.init():
253         raise Exception("GLFW can't be initialized")
254
255     window = glfw.create_window(800, 600, "lab6", None, None)
256     if not window:
257         glfw.terminate()
258         raise Exception("GLFW window can't be created")
259
260     glfw.make_context_current(window)
261
262     glfw.set_key_callback(window, key_callback)
263     glfw.set_scroll_callback(window, scroll_callback)
264     init()
265
266     while not glfw.window_should_close(window):
267         '''
268         current_time = glfw.get_time()
269         if measure_start_time is not None and current_time -
measure_start_time >= MEASURE_DURATION:
270             break
271         '''
272         update_motion()
273         update_light()
274         display()
275         glfw.poll_events()
276
277     glfw.terminate()
278
279     '''
280     if frame_count > 0:
281         avg_frame_time = (total_render_time / frame_count) * 1000
282         fps = frame_count / total_render_time
283         print(f"{frame_count}")
284         print(f"{avg_frame_time:.3f} ms")
285         print(f"FPS: {fps:.2f}")
286         '''

```

	изначально			
общее число кадров	190	184	228	216
среднее время кадра, мс	26.311	27.309	21.950	23.203
FPS	38.01	36.62	45.56	43.10
	дисплейные списки			
общее число кадров	12928	12414	12500	
среднее время кадра, мс	0.384	0.389	0.386	
FPS	2572.17	2573.52	2588.58	
	массив вершин			
общее число кадров	12501	12655	12734	12233
среднее время кадра, мс	0.386	0.382	0.380	0.395
FPS	2587.35	2616.79	2631.83	2531.84
	отсечение невидимых граней			
общее число кадров	13056	12872	12901	12549
среднее время кадра, мс	0.371	0.375	0.375	0.385
FPS	2696.17	2674.34	2666.39	2598.64
	ограничение количества вызовов пересчёта света			
общее число кадров	12217	12324	12346	12164
среднее время кадра, мс	0.363	0.359	0.359	0.351
FPS	2755.09	2784.49	2789.09	2852.09

Таблица 1: Результаты замеров, каждый из которых длился 5 с

4 Заключение

В ходе выполнения лабораторной работы была проведена оптимизация OpenGL-приложения, реализующего отрисовку параметрического эллипсоида. Основной целью являлось повышение производительности на основе применения различных методик оптимизации.

В процессе работы были реализованы следующие четыре оптимизации:

1. Дисплейные списки — использованы для хранения неизменяющейся геометрии эллипсоида, что позволило значительно снизить нагрузку на CPU за счёт предварительной компиляции команд OpenGL.
2. Массивы вершин — позволили передавать геометрические данные (вершины, индексы, текстурные координаты) в виде массивов, что существенно уменьшило количество вызовов функций и повысило эффективность передачи данных на GPU.
3. Отсечение невидимых граней — активировано с помощью стандартных средств OpenGL. Это позволило исключить из рендеринга грани, обращённые в противоположную от камеры сторону.
4. Ограничение количества вызовов пересчёта освещения — реализовано через флаг обновления освещения. Параметры источника света теперь пересчитываются только при изменении соответствующих флагов, что сократило число вызовов `glLightfv` и уменьшило накладные расходы при отрисовке.

Для оценки эффективности применённых методов была проведена серия замеров производительности, включающая измерение средней длительности одного кадра и вычисление FPS (кадров в секунду). Замеры производились в течение фиксированного интервала времени (5 секунд) до и после применения оптимизаций.

Таким образом, поставленная задача была успешно решена. Выбранные методы оптимизации являются практичными и дают значительный прирост производительности в приложениях, построенных с использованием OpenGL.